

Data types specify how & what type of data we enter into a program.

languages has some pre defined set of data types to handle various kinds of data that we have in our programs. Every data type have diff. storage capacity.

There are two types of data types.

1. Primary data types: These are fundamental data types in C namely $\&$ integer, char, float, void etc.

2. Derived data types: Derived data types are nothing but primary data types but a little twisted or grouped together like structure, union etc.

eg: student:

name - char

Rollno - int

marks - float

struct student {

int Rollno;

char name[];

float marks;

4.	Double	8 byte	%.f
5			
6	Byte	8 bits	
7	short int	2 byte	
8	long		%.l

64 bit comp.
 32 bit comp.
 size of registers.
 (fastest memory)

Register

Register is the smallest memory in comp. and only memory that can interact with ALU.

```
#include <stdio.h>
```

```
{
```

```
int main {
```

```
printf("Hello world");
```

```
return 0;
```

```
}
```

→ That meaning cannot be changed.

→ There are total of 32 keywords in C language.

eg: → Integer, float, while, do, if, else etc

Identifiers

Identifiers are the name given to variable, constants, functions name & user defined data.

There are some set of rules for writing identifiers.

Rule 1: An identifier can only have alphanumeric char (a-z, A-Z, 0-9) and underscore (—)

Rule 2: NO special character such as !, \$, @, © whitespaces etc are allowed in an identifier.

eg: abc, Abc 123, -abc

Rule 3: The first char of an identifier must be either alphabet (a-z, A-Z) or underscore. It cannot start with number.

123abc (X), =abc ✓

Rule 4: Identifiers are case sensitive.

eg: abc is different from Abc.

Rule 5: keywords are not allowed to be used as identifier.

```
// #include <stdio.h>
int main {
    return 0;
}
```

} basic structure

// command to include file of declaration of those prototypes

gcc - filename.c → compile
./a.exe → execute

2. Non-volatile memory → The purpose of this type memory is to store data permanently, even after the power is switched off the data is not lost.
eg: → hard disk.

1) The total memory capacity of a computer can be seen by hierarchy of components means a computer consist of ram, cache or some other devices.

2) Hierarchy consist of all storage devices consist in a system or available in a system, from slow auxillary memory to fast main memory to smaller cache memory.

* Comparison in terms of access time

→ Auxillary memory takes roughly 1000 times of access times than of main memory.

eg: let the access time of main is 1 unit per milli second then time taken by auxillary memory is 1 unit per second.

→ Access time of cache is roughly 7-10 times faster than main memory.

→ when the program is not in main memory but requires for execution then it is brought to main memory for auxillary memory.

→ Program which are not currently needed or not in execution are

Memory Access Methods

→ Each memory type in a collection of numerous memory locations to access data from any memory first it must be located and then the data is used/read from the memory location.

1. Random Access Memory (RAM)

Main memory are random access memory in which each memory location have unique address using this unique address any memory location can be accessed or reached in the same amount of time in any order.

2. Sequential Access

This method allows memory access in sequence or in a particular order.

3. Direct Access

combination of tracks $\rightarrow$ Platter



4. Main memory / Ram / Primary memory
The memory unit that communicates directly within the CPU. Auxiliary memory and cache memory is called main memory. It is the central storage unit of computer system. It is large and fast memory used to store data during computer operation. Various types of RAM are as follows:

1) D-RAM (dynamic RAM)

$\rightarrow$ Dynamic Ram is made up of capacitors and transistors. It must be refreshed every 10-100 milliseconds. It is slower and cheaper than S-RAM.

2) S-RAM (static RAM)

$\rightarrow$ In static RAM there are 6 transistors circuit in each cell. It retains data until power is out.

3. N-V. RAM
Non-volatile ram retains its data even when power is turned off.

RAM (Read Only Memory)
It is a non-volatile memory i.e. it stores data permanently. ROM consist of a special code called Bootstrap loader.

Bootstrap loader is a program responsible for loading an operating system at the time of computer start / turned on.

1) P. ROM 2) EP. ROM 3) EEP. ROM
are commonly used ROM

* Auxiliary Memory

Devices that provide back ups storage for permanent purpose are called auxiliary memory. Magnetic disk and magnetic tapes are some commonly used auxiliary memory.

It is not directly accessible to the CPU. It is access using the input output channels called System Calls.

* Cache Memory

The data or content of the main memory that are used again - again or frequently by CPU are stored in

the cache so that it can be easily accessed in shorter time.

→ whenever the CPU needs to access memory it first checks the cache memory, if the data is not found in cache then CPU moves to main memory.

→ it also transfer blocks of recent data into the cache and keeps on deleting the old data to accommodate the new data.

Depending upon the size of a network the network is divided into three categories:-

1. LAN
2. MAN of Autonomous Network
3. WAN

Internet is a connection of Autonomous network.

Network Topology

- BUS
- STAR
- RING
- MESH
- Tree

1. BUS



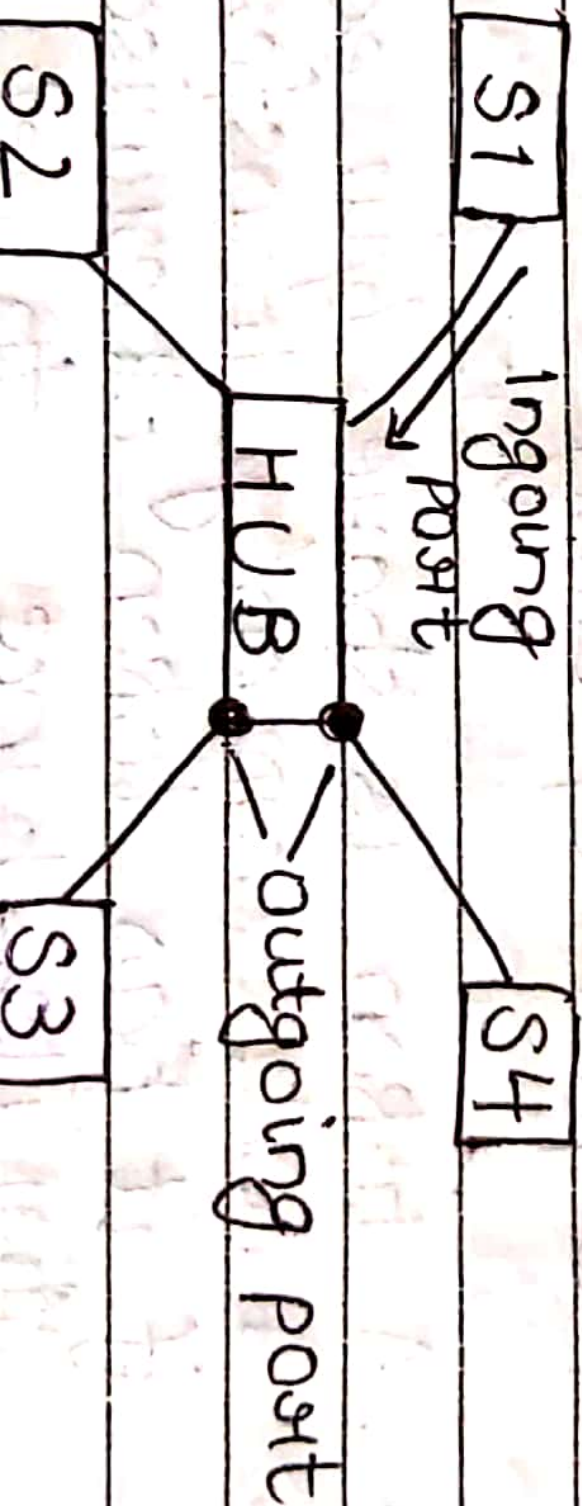
Advantages of topology

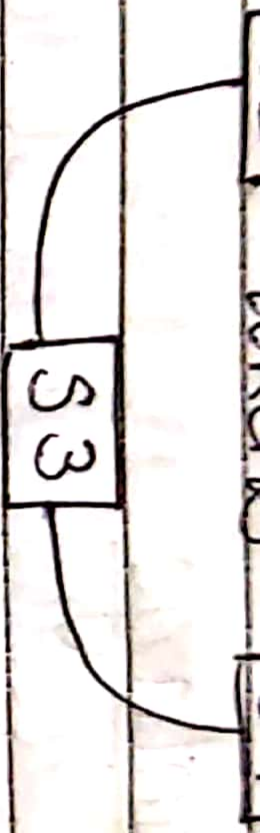
1. This topology is cost efficient.

Disadvantage

If the bus wire became unavailable for just a moment then the whole network shuts down.

Star





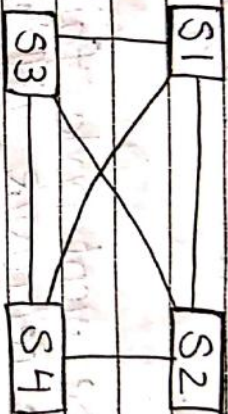
Properties:-

- The nodes connect with each other in the form of ring.
- A token is share b/w each node.
- A node can only transfer the data if it has token otherwise it will wait for its chance to get token.

be mac.

4. MESH TOPOLOGY

→ In MESH Topology every node is directly connected with each other.



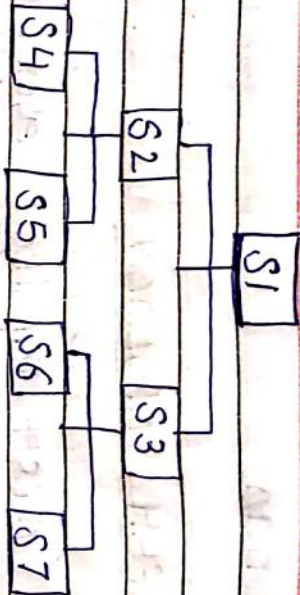
→ It is very costly.

→ Adding a new node in the network will not affect the working of the network but it is costly to add because adding of a single node can result in addition of multiple wires.

→ The data transmission delay is minimum.

5. TREE TOPOLOGY

The network will be connected in hierarchy.



→ The nodes are arranged in hierarchy form.

→ If the root node fails then the whole network will shut down, but if any leaf node fails then it will not affect the working of the network.

→ Addition of a new node is very easy.

→ This is much more efficient.

What is network Topology :-

Network topology is the arrangement of the element of a communication network. Network topology can be used to define or describe the arrangement of various types of telecommunication networks including command and control station networks, computer network and industrial field buses.

COMPUTER NETWORK & HARDWARE DEVICES

Hardware devices which are used to connect computer, printer, fax machine and other devices through a network are called network devices.

These devices transfer data in a fast, secure and correct manner over a same or different network.

Network devices may be internetwork or intranetwork.

Network Devices

Modem → It is a device that enables a computer to send & receive data on telephone or cable lines. The data stored on the computer is digital whereas a telephone line can transmit only analog data.

→ Modulator converts digital data into analog data when computer sends the data where as d-modulator converts analog data to digital data when the data is received by the computers.

tion with other devices on the network. Each ethernet card have a physical address called Mac address which never changes. It is of 48 bits.

IP address is used to identify the network. It is of 32 bits. It keeps on changing every time we connect to internet.

Port number: It is used to identify the services running on the operating system within the computer. It is of

The typical speed of ethernet card is 10^{-8} to 10^8 mbps (Mega bits per seconds)

Routers

- Routers is a network layer hardware device that transmits data from one LAN to another. If both network supports the same set of protocols.

Gateway

- Gateway is network device used to connect 2 or more dissimilar networks
- A gateway is usually computer with ~~can~~ multiple NIC connected to the different network.

Switch

- Switch is a network device that connects other devices to ethernet networks through twisted pair cables.

Wifi - card

- Wifi card is used to connect any device to the local network wirelessly. The physical area of network

user and hardware which allows user to access hardware efficiently and effectively.

O.S provides various services which are essential for the working of computer system.

Some of the services are as follows:-

1. Memory management : It is the RAM management.

2. Process management

3. File management / Disc management

4. Device management → IO operate IO calls other external devices.

read, write and execute.

* Memory Management

Management memory is the management of RAM. Operating system keeps track of available space in the main memory i.e. what part of it is in use and what part is free.

In multiprogramming the O.S. decide which process will get how much memory and RAM.

Allocation & deallocation of memory when a process request it or process no longer need it is manage by operating system.

* Process Management

In multiprogramming environment O.S. decide which process will get the processor when & for how much time.

The function is called process scheduling.

An OS does the following activity for process management.

It keeps track of processor and status of process.

how much time and when.

File management

A file management is normally organised into directory for easy usage and navigation. These directories may contain files & other directories also.

- * FAT32 → File allocation technique 32(1-block
- * NTFS → (New table file system)
- I node → Index node

1. for loop → It starts from one point and come back to the same time.

→ for doing repetitive task.

Syntax for for loop

for (; ;)

for (initialization ; condition ; exp)

{
 Body of the loop
}

2. while loop

syntax

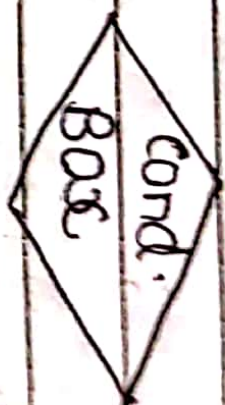
while (condition) {

}

Do while also works when the condition fails.

Infinity loop while (1) {

break;
}



Algorithms

It is a combination of sequence of finite steps to solve a particular problem.

Write an algo. for printing a factorial of n .

2. while
3. print fact.
4. END

Switch

```
if (a==1)
    a=5;
else if (a==2)
    a=12;
else if (a==3)
    a=4;
else if (a==4)
    a=1;
switch (a) {
    case 1: a=5; break;
    case 2: a=12;
    case 3: a=4;
    case 4: a=1;
}
```

int arr[10];

↓ name [size];

Data type

Array takes constant time i.e $O(1)$



0 1 2 3 4

$$4000 + 2(4)$$

$$= 4008$$

Base add + index * size of (data type)

int arr[10] = {0}; all value 0

In case of excessive arr lower size we can initialize like static memory Allocation (compilation access memory)

int n;

scanf("%d", &n);

int arry[n];

Date 14/10/19

2D Array

```
#include <stdio.h>
int main() {
    int arr[6][6];
    for(i=0; i<6; i++)
        for(j=0; j<6; j++)
            scanf("%d", &arr[i][j]);
    for(i=0; i<6; i++)
        for(j=0; j<6; j++)
            scanf("%d", &arr[i][j]);
    printf("\n");
    return 0;
```

Base address

- * $\text{arr} + i * \text{no. of col.} + j * \text{size of data}$
- * $\text{arr} + j * \text{no. of col.} + i * \text{size of data}$

Stack $\rightarrow$ local variable store
heap $\rightarrow$ dynamic.

arr = (int**) malloc (row * size of int*)

comparisions

No. of comparisions

$$\Rightarrow \frac{n(n-1)}{2}$$

n is no. of element

00
01
02
10
11
12
20
21
22

Sorting

Arranging the numbers in some order in sorting.

eg:- 64, 63, 7, 2, 48

Output 2, 7, 48, 63, 64

Algorithms for sorting

1. selection sort
2. Bubble sort
3. Insertion sort
4. Merge sort
5. Quick sort
6. counting sort
7. Radix sort
8. Shell sort
9. Bucket sort
10. Heap sort.

for 10^9 no. $\rightarrow$ counting sort.

1. selection sort

[7 | 3 | 8 | 2 | 5]

0 1 2 3 4

1st pass [2, 7, 3, 8, 5]

2nd pass [2, 3, 7, 8, 5]

3rd pass [2, 3, 5, 8, 7]

Output 4th pass [2, 3, 5, 7, 8]

Stable sort :- If implemented carefully.

$\Rightarrow$ Orig $\rightarrow 7, 3a, 2, 3b, 5$

$\Rightarrow$ Sorted $\rightarrow 2, 3a, 3b, 5, 7$

count no. of swaps

$s=0$

for $(i=0; i < n-1; i++)$ {

for $(j=i+1; j < n; j++)$ {

}

count no. of comparison

$$\frac{n(n-1)}{2} + n - 1 \quad (:j=i)$$

Bubble Sort

Monday 4/11/19

7	3	8	9	0	3
0	1	2	3	4	5

After each iteration largest no. reaches to the end.

Syntax:

for ($i=0$; $i < n$; $i++$)

{
for ($j=0$; $j < n-i$; $j++$)

{
if ($arr[j] > arr[j+1]$)

{
temp = arr[j];
arr[j] = arr[j+1];
arr[j+1] = temp;

}

}

Bubble

2. After i^{th} iteration $(i+1)^{\text{th}}$ largest element will be at i^{th} $(n-i-1)^{\text{th}}$ position
- complexity comparison.

	Selection	Bubble	Insertion
worst	$O(n^2)$	$O(n^2)$	$O(n^2)$
Best	$O(n^2)$	$O(n^2)$	$O(n)$
Avg	$O(n^2)$	$O(n^2)$	$O(n^2)$

In case of comparison bubble sort takes less comp. thus it is preferable over selection.

```

#include <stdio.h>

int main()
{
    scanf("%d", &n);
    int arr[n], i, j, temp, key;

    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    for (i = 0; i < n; i++) {
        key = arr[i];
        j = i + 1;
        while (j < n && arr[j] > key)
            arr[j + 1] = arr[j];
        j--;
        arr[j + 1] = key;
    }
}

```

Complexity list Date

	Selection	Bubble	Insertion
worst	$O(n^2)$	$O(n^2)$	$O(n^2)$
Avg	$O(n^2)$	$O(n^2)$	$O(n^2)$
Best	$O(n^2)$	$O(n^2)$	$O(n)$
space	constant	$O(1)$	$O(1)$

inplace algorithm

Segmentation fault
 whenever we (program) try to access
 illegal memory at run time segmentation
 fault occur.

Bound checking
 it checks the boundary.

Auto : It automatically assigns the variable to Ram & register.

Syntax : Auto int a;

extern : extern variable can be used from one file to another file.

Static : Static variable keep up the calculated value even after the function gets destroyed. Initialization takes place once & compulsory.

eg : static int i = 0; // initialization

Constant : cons. int a = 10;

It keeps / sets the value permanently in the program & refuse change to the value.

return type function name (Arguments)

```
int main()  
{  
    int a, b, sum;  
    scanf ("%d %d", &a, &b);  
    sum = add(a, b);  
    printf ("%d", sum);  
}  
int add(int x, int y)  
{  
    return x + y;  
}
```

Q. WAF function which will multiply a float & int.

```
=> float mult (float x, int y)  
{  
    return x * y;  
}
```

Q. WAF function which given a, b, c prints the two possible value of x, y such value does not exist the function should print imaginary value.

MON

Date 11/11/19

Functions

Function are the set of code which take inputs, and do some specific task and give output.

Declaration of function

```
Returntype functionName(parameters) {  
    }  
Body of function
```

* Reusability

* Error detection & correction become easy.

1. Prototype or Signature

2. Definition

```
ex: #include <stdio.h>
```

```
void sum();
```

```
int main() {
```

```
    sum();
```

```
    return 0;
```

```
}
```

```
void sum() {
```

```
    int a, b;
```

```
    scanf("%d %d", &a, &b);
```

```
    printf("%d", a+b);
```

```
}
```

Date: Friday

15/11/19

Memory Representation of C Program Variables

Ram	OS reserved memory	→ operating system
	stack	→ local var, function parameters & function calls.
	↓	
	shared libraries	
(grow upwards)	Heap	Dynamic variables
	Read/write memory	→ global variables
	Read only memory	→ constants.

It is volatile type memory, i.e. RAM.

How compilers differentiate b/w global, local, static variables?

- * Illegal memory access by a program causes segmentation fault.
- * stack is data structure is used for function call, Queue is used for process scheduling.
- * ROM is used to store bootstrap loader. (read only memory)
- * It is a type of non volatile memory.
- * Shared libraries.



It is shared once so that other programs can use it.
Queue is used by shared library.

```
or
•
#include <stdio.h>           not stored
int a = 10; #define PI 3.14
int a = 10;      R/W memory
int main() {
    const int b = 20; #read only
    int c = 10; #stack
    display(c);
    return 0;
}

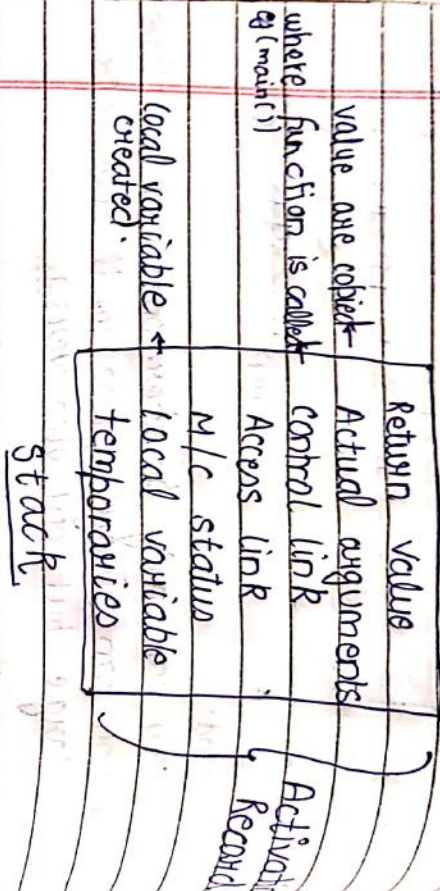
void display (int p) { #stack
    int q = 10; #stack
}
```

* Static variables are not destroyed after the function ends, it remains the same and keeps the updated value.

21/11/19

Tuesday

Activation Record



* A function gets activated only when the function is called!

⇒ M/c status (machine) → checking of status of machine before calling a function.

⇒ The temporary variable used for functioning are stored in temporaries.

→ Access link is created between the function & where it is called, so that the function can access the data from it.

* Control stack is a runtime stack which is used to keep track of life process activation.

i.e. It is used to find out the procedures whose execution have not been completed.

- * When the activation begins then the procedure name will push onto the stack.

- * When activation ends then it will be **POP**.

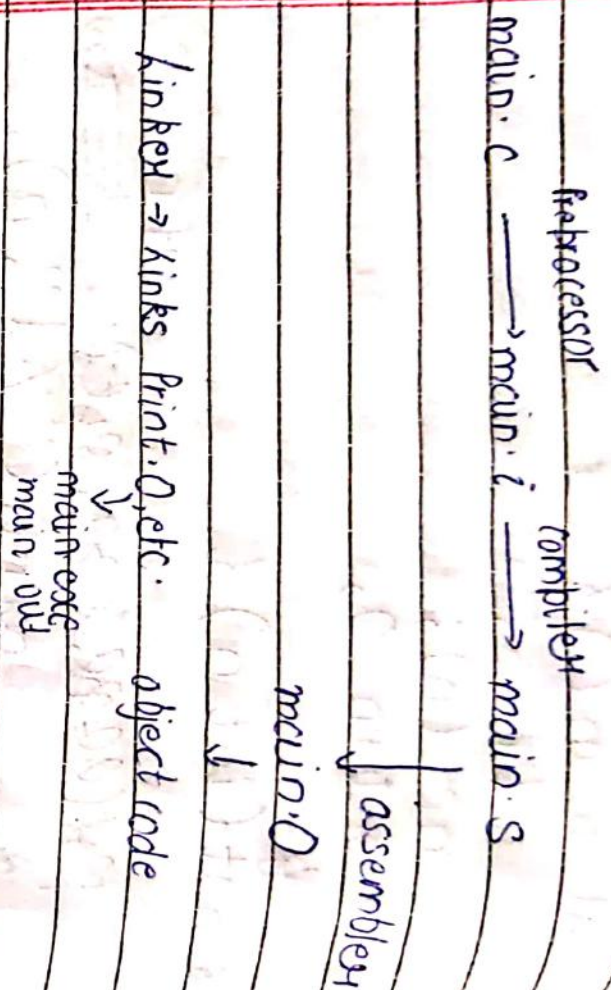
- * Activation record is used to manage the information needed by the single execution of a procedure.

= Returning It is used by calling procedure to ~~return~~ the message from callee procedure.

⇒ Actual Parameter → It is used by callee procedure to substitute parameters to the called procedure.

⇒ Control link = It points to the activation record of caller.

⇒ Access link ⇒ It is used to refer to non local data or global data or other activation records.



linker does static & dynamic linking

Static linking

When the program is compiled the compiler generates object code after generating the object code. The compiler also invokes linker. One of the main task for linker is to make code of library functions available at your program. Linker can accomplish this task in two ways.

1. By copying the code of library functions to your object code. (static linking)

11/19

Date

at runtime. (dynamic linking)

Indirect Recursion

INDIRECT RECURSION

Direct Recursion: void foo() {
 foo();
}

Indirect Recursion: void foo() {
 { base cond: }
 fun();
}

void fun() {
 { base cond: }
 foo();
}

25/11/19

Monday

Dynamic Memory allocation → allocated at run time.

Static Memory allocation → allocated at compile time.

Dynamic memory is extensively used in linked list, trees, graphs.

Pointers int a[10];

Pointer is used to store address.

declaration: int *a; { holding address